Experiment 3

Theory

Natural Language Processing (NLP) is a branch of Artificial Intelligence (AI) that enables computers to understand, interpret, and process human language. Before performing tasks such as text classification, keyword extraction, sentiment analysis, or machine learning, textual data must be preprocessed. Text preprocessing improves the quality of text by removing unnecessary information and converting it into a structured format suitable for analysis.

In this experiment, the preprocessing techniques **Stop Word Removal**, **Lemmatization**, and **Stemming** are applied to resume text for keyword extraction.

Stop Word Removal

Stop words are commonly occurring words in a language that carry little or no meaningful information for text analysis. Examples include **"the," "is," "and," "with," "in,"** and **"of."** Removing stop words reduces the size of the text and helps focus on important words that contribute to the meaning of the document.

Example

Original Sentence:

> Experienced Python developer with expertise in machine learning and data analysis.

After Stop Word Removal:

> Experienced Python developer expertise machine learning data analysis.

Lemmatization

Lemmatization is the process of converting words into their base or dictionary form, known as the **lemma**. It considers the meaning and grammatical context of a word, producing valid dictionary words. This technique helps normalize text while preserving its meaning.

Examples

Original Word	Lemmatized Word
studies	study
cars	car
running	run
developers	developer

Stemming

Stemming is the process of reducing words to their root form by removing prefixes and suffixes using predefined rules. Unlike lemmatization, stemming does not consider the meaning of words and may produce non-dictionary words. However, it is faster and commonly used in information retrieval systems.

Examples

Original Word	Stemmed Word
learning	learn
developer	develop
experienced	experienc
analysis	analysi

Importance of Text Preprocessing

Text preprocessing is an essential step in Natural Language Processing because it:

- Removes unnecessary words and symbols.
- Reduces the size of the vocabulary.
- Improves the efficiency and accuracy of NLP algorithms.
- Standardizes text for better analysis.
- Enhances keyword extraction and document classification.

Application in Resume Keyword Extraction

In resume keyword extraction, preprocessing techniques help identify important skills and technical terms such as **Python**, **Machine Learning**, **SQL**, **Data Analysis**, and **NLP**. Removing stop words and reducing words to their base forms improves the accuracy of identifying relevant keywords used in resume screening and recruitment systems.

In [17]:
```
!pip install nltk
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: nltk in /home/computer/.local/lib/python3.10/site-packages (3.10.0)
Requirement already satisfied: joblib in /home/computer/.local/lib/python3.10/site-packages (from nltk)
(1.5.3)
Requirement already satisfied: tqdm in /home/computer/.local/lib/python3.10/site-packages (from nltk) (4.
69.0)
Requirement already satisfied: defusedxml in /home/computer/.local/lib/python3.10/site-packages (from nlt
k) (0.7.1)
Requirement already satisfied: regex>=2021.8.3 in /home/computer/.local/lib/python3.10/site-packages (fro
m nltk) (2026.7.19)
Requirement already satisfied: click in /usr/lib/python3/dist-packages (from nltk) (8.0.3)
```

In [18]:
```
import nltk


nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

```
[nltk_data] Downloading package punkt to /home/computer/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     /home/computer/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /home/computer/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /home/computer/nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
```

Out[18]:
```
True
```

In [6]:
```
df = pd.read_csv("UpdatedResumeDataSet.csv")

df.head()
```

Out[6]:

	Category	Resume
0	Data Science	Skills * Programming Languages: Python (pandas...
1	Data Science	Education Details \r\nMay 2013 to May 2017 B.E...
2	Data Science	Areas of Interest Deep Learning, Control Syste...
3	Data Science	Skills â¯¢ R â¯¢ Python â¯¢ SAP HANA â¯¢ Table...
4	Data Science	Education Details \r\n MCA YMCAUST, Faridab...

In [7]:
```
resume = df.loc[0, "Resume"]

print(resume[:200] + "...")
```

```
Skills * Programming Languages: Python (pandas, numpy, scipy, scikit-learn, matplotlib), Sql, Java, JavaS
cript/JQuery. * Machine learning: Regression, SVM, NaÃ¯ve Bayes, KNN, Random Forest, Decision T...
```

In [8]:
```
resume = resume.lower()

print(resume[:200] + "...")
```

```
skills * programming languages: python (pandas, numpy, scipy, scikit-learn, matplotlib), sql, java, javas
cript/jquery. * machine learning: regression, svm, naã¯ve bayes, knn, random forest, decision t...
```

In [9]:
```
tokens = word_tokenize(resume)

print(tokens[:20])
```

```
['skills', '*', 'programming', 'languages', ':', 'python', '(', 'pandas', ',', 'numpy', ',', 'scipy',
',', 'scikit-learn', ',', 'matplotlib', ')', ',', 'sql', ',']
```

In [10]:
```
stop_words = set(stopwords.words("english"))

filtered_words = [word for word in tokens if word.isalnum() and word not in stop_words]

print(filtered_words[:20])
```

```
['skills', 'programming', 'languages', 'python', 'pandas', 'numpy', 'scipy', 'matplotlib', 'sql', 'java',
'machine', 'learning', 'regression', 'svm', 'bayes', 'knn', 'random', 'forest', 'decision', 'trees']
```

In [11]:
```
lemmatizer = WordNetLemmatizer()

lemmatized_words = [lemmatizer.lemmatize(word) for word in filtered_words]

print(lemmatized_words[:20])
```

```
['skill', 'programming', 'language', 'python', 'panda', 'numpy', 'scipy', 'matplotlib', 'sql', 'java', 'm
achine', 'learning', 'regression', 'svm', 'bayes', 'knn', 'random', 'forest', 'decision', 'tree']
```

In [12]:
```
stemmer = PorterStemmer()

stemmed_words = [stemmer.stem(word) for word in filtered_words]

print(stemmed_words[:20])
```

```
['skill', 'program', 'languag', 'python', 'panda', 'numpi', 'scipi', 'matplotlib', 'sql', 'java', 'machi
n', 'learn', 'regress', 'svm', 'bay', 'knn', 'random', 'forest', 'decis', 'tree']
```

In [14]:
```
comparison = pd.DataFrame({
    "Original": filtered_words[:20],
    "Lemmatized": lemmatized_words[:20],
    "Stemmed": stemmed_words[:20]
})

comparison
```

Out[14]:

	Original	Lemmatized	Stemmed
0	skills	skill	skill
1	programming	programming	program
2	languages	language	languag
3	python	python	python
4	pandas	panda	panda
5	numpy	numpy	numpi
6	scipy	scipy	scipi
7	matplotlib	matplotlib	matplotlib
8	sql	sql	sql
9	java	java	java
10	machine	machine	machin
11	learning	learning	learn
12	regression	regression	regress
13	svm	svm	svm
14	bayes	bayes	bay
15	knn	knn	knn
16	random	random	random
17	forest	forest	forest
18	decision	decision	decis
19	trees	tree	tree

In [ ]: